Data Structures

HomeWork Set -7

1. Construct an Expression Tree for a Given Expression

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

struct Node {
   char data;
   struct Node *left, *right;
};

struct Node* createNode(char ch) {
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));

   newNode->data = ch;
   newNode->left = NULL;
   newNode->right = NULL;

   return newNode;
}

void preorder(struct Node *root) {
   if (root != NULL) {
      printf("%c ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void inorder(struct Node *root) {
   if (root != NULL) {
      if (root->left != NULL)
         printf("(");

      inorder(root->left);
      printf("%c", root->data);
      inorder(root->right);

      if (root->right != NULL)
         printf(")");
   }
}
```

```
int main() {
    char postfix[100];
    struct Node *stack[100];
    int top = -1, i;

    printf("Enter postfix expression: ");
    scanf("%99s", postfix);

    for (i = 0; postfix[i] != '\0'; i++) {
        char ch = postfix[i];

        if (isalnum((unsigned char)ch)) {
            stack[++top] = createNode(ch);
        }
        else {
            if (top < 1) {
                printf("Invalid postfix expression\n");
                return 1;
            }

            struct Node *newNode = createNode(ch);

            newNode->right = stack[top--];
            newNode->left = stack[top--];

            stack[++top] = newNode;
        }
    }

    if (top != 0) {
        printf("Invalid postfix expression\n");
        return 1;
    }

    struct Node *root = stack[top];

    printf("Inorder expression: ");
    inorder(root);

    printf("\nPreorder traversal: ");
    preorder(root);

    printf("\n");

    return 0;
}
```

Output :

```
Enter postfix expression: ab-cd*+
Inorder expression: (a-b)+(c*d)
Preorder traversal: + - a b * c d
```

2: Check whether a Binary Tree is a Binary Search Tree (BST).

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* createTree() {
   int value;

   scanf("%d", &value);

   if (value == -1)
      return NULL;

   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));

   newNode->data = value;

   printf("Enter left child of %d (-1 for none): ",
         value);
   newNode->left = createTree();

   printf("Enter right child of %d (-1 for none): ",
         value);
   newNode->right = createTree();

   return newNode;
}

int isBST(struct Node *root, long long min,
         long long max) {
   if (root == NULL)
      return 1;

   if (root->data <= min || root->data >= max)
      return 0;

   return isBST(root->left, min, root->data) &&
```

```
        isBST(root->right, root->data, max);
}

int main() {
   struct Node *root;

   printf("Enter root value (-1 for empty tree): ");
   root = createTree();

   if (isBST(root, LLONG_MIN, LLONG_MAX))
      printf("The binary tree is a BST.\n");
   else
      printf("The binary tree is NOT a BST.\n");

   return 0;
}
```

Output :

```
Enter root value (-1 for empty tree): 40
Enter left child of 40 (-1 for none): 20
Enter left child of 20 (-1 for none): 10
Enter left child of 10 (-1 for none): -1
Enter right child of 10 (-1 for none): -1
Enter right child of 20 (-1 for none): 30
Enter left child of 30 (-1 for none): -1
Enter right child of 30 (-1 for none): -1
Enter right child of 40 (-1 for none): 60
Enter left child of 60 (-1 for none): -1
Enter right child of 60 (-1 for none): -1
The binary tree is a BST.
```

3: Implement a Max Heap in C.

```
#include <stdio.h>

int heap[100];
int size = 0;

void insert(int value) {
   int i, parent, temp;

   if (size == 100) {
      printf("Heap is full!\n");
      return;
   }

   i = size;
   heap[size] = value;
```

```
    size++;

    // Move the new element upward
    while (i > 0) {
        parent = (i - 1) / 2;

        if (heap[parent] >= heap[i])
            break;

        temp = heap[parent];
        heap[parent] = heap[i];
        heap[i] = temp;

        i = parent;
    }
}

void display() {
    int i;

    printf("Max Heap: ");
    for (i = 0; i < size; i++)
        printf("%d ", heap[i]);

    printf("\n");
}

int main() {
    int n, value, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n < 0 || n > 100) {
        printf("Invalid number of elements.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        printf("Enter element: ");
        scanf("%d", &value);
        insert(value);
    }

    display();

    return 0;
}
```

Output :

```
Enter number of elements: 5
Enter element: 10
Enter element: 30
Enter element: 20
Enter element: 50
Enter element: 40
Max Heap: 50 40 20 10 30
```

4: Check Whether an Undirected Graph Contains a Cycle.

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int visited[MAX];
int n;

int hasCycleDFS(int vertex, int parent) {
    int i;

    visited[vertex] = 1;

    for (i = 0; i < n; i++) {
        if (graph[vertex][i]) {
            if (!visited[i]) {
                if (hasCycleDFS(i, vertex))
                    return 1;
            }
            else if (i != parent) {
                return 1;
            }
        }
    }

    return 0;
}

int main() {
    int edges, u, v, i;
    int cycle = 0;

    printf("Enter number of vertices and edges: ");
    scanf("%d %d", &n, &edges);

    if (n < 1 || n > MAX || edges < 0) {
        printf("Invalid input.\n");
        return 1;
    }
```

```
    printf("Enter edges (vertices numbered 0 to %d):\n",
           n - 1);

    for (i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);

        if (u < 0 || u >= n || v < 0 || v >= n) {
            printf("Invalid edge.\n");
            return 1;
        }

        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    for (i = 0; i < n; i++)
        visited[i] = 0;

    for (i = 0; i < n; i++) {
        if (!visited[i]) {
            if (hasCycleDFS(i, -1)) {
                cycle = 1;
                break;
            }
        }
    }

    if (cycle)
        printf("Cycle exists.\n");
    else
        printf("No cycle exists.\n");

    return 0;
}
```

Output :

```
Enter number of vertices and edges: 3 3
Enter edges (vertices numbered 0 to 2):
0 1
1 2
2 0
Cycle exists.
```

5: Count the Number of Islands in a Grid.

```
#include <stdio.h>
```

```
#define ROW 5
#define COL 5

char grid[ROW][COL] = {
   {'L', 'L', 'W', 'W', 'W'},
   {'W', 'L', 'L', 'W', 'W'},
   {'L', 'W', 'W', 'L', 'L'},
   {'W', 'W', 'W', 'W', 'W'},
   {'L', 'W', 'L', 'L', 'W'}
};

int visited[ROW][COL];

int dx[] = {-1, -1, -1, 0, 0, 1, 1, 1};
int dy[] = {-1,  0,  1,-1, 1,-1, 0, 1};

void DFS(int r, int c) {
   int k, nr, nc;

   visited[r][c] = 1;

   for (k = 0; k < 8; k++) {
      nr = r + dx[k];
      nc = c + dy[k];

      if (nr >= 0 && nr < ROW &&
         nc >= 0 && nc < COL &&
         grid[nr][nc] == 'L' &&
         !visited[nr][nc]) {
         DFS(nr, nc);
      }
   }
}

int main() {
   int r, c, islands = 0;

   for (r = 0; r < ROW; r++) {
      for (c = 0; c < COL; c++) {
         if (grid[r][c] == 'L' &&
            !visited[r][c]) {
            islands++;
            DFS(r, c);
         }
      }
   }

   printf("Number of islands: %d\n", islands);

   return 0;
```

```
}
```

Output :

Number of islands: 4

6: Find the Minimum Spanning Tree using Kruskal's Algorithm.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Edge {
   int src, dest, weight;
};

int parent[MAX];
int rankSet[MAX];

int find(int x) {
   if (parent[x] != x)
      parent[x] = find(parent[x]);

   return parent[x];
}

void unionSets(int x, int y) {
   int rootX = find(x);
   int rootY = find(y);

   if (rootX == rootY)
      return;

   if (rankSet[rootX] < rankSet[rootY]) {
      parent[rootX] = rootY;
   }
   else if (rankSet[rootX] > rankSet[rootY]) {
      parent[rootY] = rootX;
   }
   else {
      parent[rootY] = rootX;
      rankSet[rootX]++;
   }
}

int compare(const void *a, const void *b) {
   const struct Edge *e1 = a;
   const struct Edge *e2 = b;
```

```
    if (e1->weight < e2->weight)
        return -1;
    if (e1->weight > e2->weight)
        return 1;
    return 0;
}

int main() {
    struct Edge edges[MAX];
    int V, E, i, count = 0;
    int totalWeight = 0;

    printf("Enter number of vertices and edges: ");
    scanf("%d %d", &V, &E);

    if (V < 1 || V > MAX || E < 0 || E > MAX) {
        printf("Invalid input.\n");
        return 1;
    }

    printf("Enter each edge: source destination weight\n");

    for (i = 0; i < E; i++) {
        scanf("%d %d %d",
              &edges[i].src,
              &edges[i].dest,
              &edges[i].weight);

        if (edges[i].src < 0 || edges[i].src >= V ||
            edges[i].dest < 0 || edges[i].dest >= V) {
            printf("Invalid vertex.\n");
            return 1;
        }
    }

    for (i = 0; i < V; i++) {
        parent[i] = i;
        rankSet[i] = 0;
    }

    qsort(edges, E, sizeof(struct Edge), compare);

    printf("Edges in MST:\n");

    for (i = 0; i < E && count < V - 1; i++) {
        int u = edges[i].src;
        int v = edges[i].dest;

        if (find(u) != find(v)) {
```

```
            printf("%d - %d : %d\n",
                  u, v, edges[i].weight);

            unionSets(u, v);
            totalWeight += edges[i].weight;
            count++;
        }
    }

    if (count != V - 1) {
        printf("MST does not exist; graph is disconnected.\n");
    }
    else {
        printf("Total weight of MST: %d\n", totalWeight);
    }

    return 0;
}
```

Output :

```
Enter number of vertices and edges: 4 5
Enter each edge: source destination weight
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total weight of MST: 19
```

7: Dijkstra's Algorithm to Find the Shortest Path from a Source to All Nodes.

```
#include <stdio.h>

#define MAX 100
#define INF 1000000000

int main() {
    int graph[MAX][MAX];
    int dist[MAX], visited[MAX];
    int V, source, i, j, count, u, v;

    printf("Enter number of vertices: ");
    scanf("%d", &V);
```

```
if (V < 1 || V > MAX) {
    printf("Invalid number of vertices.\n");
    return 1;
}

printf("Enter adjacency matrix:\n");

for (i = 0; i < V; i++) {
    for (j = 0; j < V; j++) {
        scanf("%d", &graph[i][j]);

        if (graph[i][j] < 0) {
            printf("Negative weights are not allowed.\n");
            return 1;
        }
    }
}

printf("Enter source vertex: ");
scanf("%d", &source);

if (source < 0 || source >= V) {
    printf("Invalid source vertex.\n");
    return 1;
}

for (i = 0; i < V; i++) {
    dist[i] = INF;
    visited[i] = 0;
}

dist[source] = 0;

for (count = 0; count < V; count++) {
    int minDist = INF;
    u = -1;

    for (i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < minDist) {
            minDist = dist[i];
            u = i;
        }
    }

    if (u == -1)
        break;

    visited[u] = 1;
```

```
        for (v = 0; v < V; v++) {
            if (!visited[v] &&
                graph[u][v] > 0 &&
                dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Shortest distances from source %d:\n", source);

    for (i = 0; i < V; i++) {
        if (dist[i] == INF)
            printf("To %d: Unreachable\n", i);
        else
            printf("To %d: %d\n", i, dist[i]);
    }

    return 0;
}
```

Output :

```
Enter number of vertices: 4
Enter adjacency matrix:
0 4 1 0
4 0 2 1
1 2 0 5
0 1 5 0
Enter source vertex: 0
Shortest distances from source 0:
To 0: 0
To 1: 3
To 2: 1
To 3: 4
```

8. Write a C program to find the mother vertex in a directed graph.

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int visited[MAX];
int V;

void DFS(int v)
{
    visited[v] = 1;
```

```
    for (int i = 0; i < V; i++)
    {
        if (graph[v][i] == 1 && !visited[i])
            DFS(i);
    }
}

int main()
{
    int candidate = -1;
    int flag = 1;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    // Find a possible mother vertex
    for (int i = 0; i < V; i++)
    {
        if (!visited[i])
        {
            DFS(i);
            candidate = i;
        }
    }

    // Check whether the candidate reaches all vertices
    for (int i = 0; i < V; i++)
        visited[i] = 0;

    if (candidate != -1)
        DFS(candidate);

    for (int i = 0; i < V; i++)
    {
        if (!visited[i])
        {
            flag = 0;
            break;
        }
    }

    if (flag && candidate != -1)
        printf("Mother vertex: %d\n", candidate);
```

```
    else
        printf("No mother vertex exists.\n");

    return 0;
}
```

Output :

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 0 0 0
0 0 1 0 0
1 0 0 1 0
0 0 0 0 1
0 0 0 0 0
Mother vertex: 0
```

9: Find all Strongly Connected Components (SCCs) in a directed graph using C.

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int transpose[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int V;

void DFS1(int v)
{
    visited[v] = 1;

    for (int i = 0; i < V; i++)
    {
        if (graph[v][i] && !visited[i])
            DFS1(i);
    }

    stack[++top] = v;
}

void DFS2(int v)
{
    visited[v] = 1;
    printf("%d ", v);

    for (int i = 0; i < V; i++)
```

```
    {
        if (transpose[v][i] && !visited[i])
            DFS2(i);
    }
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
        {
            scanf("%d", &graph[i][j]);
            transpose[j][i] = graph[i][j];
        }

    // First DFS: store vertices by finishing order
    for (int i = 0; i < V; i++)
        visited[i] = 0;

    for (int i = 0; i < V; i++)
        if (!visited[i])
            DFS1(i);

    // Reset visited
    for (int i = 0; i < V; i++)
        visited[i] = 0;

    // Second DFS on the transposed graph
    printf("Strongly Connected Components:\n");

    while (top >= 0)
    {
        int v = stack[top--];

        if (!visited[v])
        {
            DFS2(v);
            printf("\n");
        }
    }

    return 0;
}
```

Output :

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 0 0 0
0 0 1 0 0
1 0 0 1 0
0 0 0 0 1
0 0 0 1 0
Strongly Connected Components:
0 1 2
3 4
```

10: Check whether a graph is Bipartite using C.

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int color[MAX];
int queue[MAX];
int V;

int isBipartite()
{
   for (int i = 0; i < V; i++)
      color[i] = -1;

   // Handle disconnected graphs too
   for (int start = 0; start < V; start++)
   {
      if (color[start] != -1)
         continue;

      int front = 0, rear = 0;

      queue[rear++] = start;
      color[start] = 0;

      while (front < rear)
      {
         int u = queue[front++];

         for (int v = 0; v < V; v++)
         {
            if (graph[u][v] == 1)
            {
               if (color[v] == -1)
               {
                  color[v] = 1 - color[u];
                  queue[rear++] = v;
```

```
                }
                else if (color[v] == color[u])
                {
                    return 0;
                }
            }
        }
    }

    return 1;
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    if (isBipartite())
        printf("Graph is Bipartite.\n");
    else
        printf("Graph is not Bipartite.\n");

    return 0;
}
```

Output :

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
Graph is Bipartite.
```